

On-Premise LLM Deployment for Enterprises

Executive Summary

Organizations adopting Large Language Models (LLMs) at scale face escalating operational expenditure driven by token-based pricing and vendor lock-in. A 10–20 user team can easily incur \$500–\$1,500/month with a single provider, with costs scaling non-linearly as usage increases.

This proposal outlines a business opportunity to deploy on-premise LLM infrastructure using open-weight models and optimized inference frameworks (e.g., SGLang), enabling companies to:

- Reduce recurring API costs
 - Maintain data privacy and sovereignty
 - Access multiple models via intelligent routing
 - Achieve predictable performance and cost control
-

Problem Statement

1. Cost Explosion

Token-based pricing introduces:

- Unpredictable billing
- High marginal cost per query
- Vendor dependency

2. Data Sensitivity

Companies hesitate to send:

- Proprietary data
- Internal communications
- Customer information

3. Vendor Lock-In

- Limited flexibility
 - Lack of model diversity
 - Dependency on API uptime and policy changes
-

Proposed Solution

Deploy local LLM infrastructure tailored to company needs.

Core Components

1. Hardware Layer

2. Entry: Single high-spec Mac Mini (~\$2,000–\$2,500)
3. Scale: Multi-node GPU clusters (NVIDIA/AMD) for larger orgs

4. Inference Layer

5. SGLang or equivalent for optimized inference
6. Quantized models (4-bit, 8-bit) for efficiency

7. Model Layer

8. Mix of open models (e.g., LLaMA variants, Mistral, etc.)
9. Task-specific specialization

10. Routing Layer (Key Differentiator)

11. Prompt classifier + router
12. Selects optimal model based on:
 - Task type (coding, summarization, reasoning)
 - Latency requirements
 - Context size

13. Interface Layer

14. Internal API (OpenAI-compatible)
 15. Web UI / chat interface
-

Key Technical Challenge: Concurrency Bottlenecks

Why This Happens

LLMs are compute-bound systems. Bottlenecks arise from:

- Limited GPU/CPU throughput
- Memory bandwidth constraints
- Sequential token generation

How Cloud Providers Solve It

1. Massive Parallelization

2. Distributed GPU clusters
 3. Tensor parallelism & pipeline parallelism
 4. **Request Batching**
 5. Combine multiple requests into a single forward pass
 6. **KV Cache Optimization**
 7. Reuse attention states to reduce recomputation
 8. **Load Balancing**
 9. Route traffic across multiple machines
 10. **Speculative Decoding**
 11. Use smaller models to accelerate larger ones
-

How We Solve Concurrency Locally

Phase 1 (Single Machine Setup)

- Use SGLang batching
- Limit concurrent sessions (e.g., 5–10 users active)
- Implement request queue with prioritization
- Use smaller/faster models for most tasks

Phase 2 (Enhanced Setup)

- Add second machine (horizontal scaling)
- Introduce load balancer
- Separate models by task (e.g., code vs chat)

Phase 3 (Advanced Optimization)

- Quantization + model distillation
 - Speculative decoding (small + large model pairing)
 - GPU acceleration (if budget allows)
-

Economic Model

Cost Comparison (Example)

Setup	Monthly Cost	Notes
API-based	\$500–\$1500	Recurring, scales with usage
Local (Year 1)	~\$2500–\$5000	Hardware + setup
Local (Year 2+)	~\$0–\$500	Maintenance only

Break-even: ~3–6 months

Revenue Model

1. Initial Setup Fee

2. \$1,000–\$5,000 depending on complexity

3. Hardware Margin

4. Optional markup or procurement fee

5. Monthly Support

6. \$200–\$1,000/month

7. Upsells

8. Fine-tuning

9. Custom routing logic

10. Integration with internal systems

Risks & Constraints

1. Performance Gap

- Open models may underperform frontier APIs

2. Hardware Limits

- Mac Mini is insufficient for high concurrency

3. Maintenance Overhead

- Updates, monitoring, debugging

4. Power Users

- Heavy usage may still require hybrid cloud fallback
-

Strategic Advantage

- Cost control vs usage-based billing
 - Data privacy (strong selling point)
 - Customizability (routing layer is key moat)
 - Vendor independence
-

Go-To-Market Strategy

Target Customers

- SMBs (10–100 employees)
- Legal, finance, healthcare (data-sensitive sectors)
- Tech startups optimizing cost

Entry Strategy

- Pilot deployment for 1 team
 - Demonstrate cost savings within 30 days
 - Expand across organization
-

Verdict: Is This Viable?

Yes — with constraints.

This is not a "replace OpenAI" play. This is a **cost-optimization + privacy + control** play.

Best positioned as:

"Private AI Infrastructure for Teams"

Success depends on:

- Efficient routing layer (critical)
 - Managing concurrency expectations
 - Smart hardware scaling strategy
-

Recommended Next Steps

1. Build MVP:

2. Mac Mini + SGLang + 2-3 models
 3. Simple routing logic
 4. Test internally or with 1 pilot company
 5. Measure:
 6. Latency
 7. Throughput
 8. Cost savings
 9. Iterate on:
 10. Concurrency handling
 11. Model selection
-

Final Take

Your instinct is directionally correct.

But the bottleneck you identified (concurrency) is not a minor issue — it is THE problem.

If you solve that well enough for SMB workloads, you have a business. If you ignore it, this collapses under real usage.

Focus there.